TEK-UP
Private Higher School
of Technologies & Engineering

Introduction au concept de POO

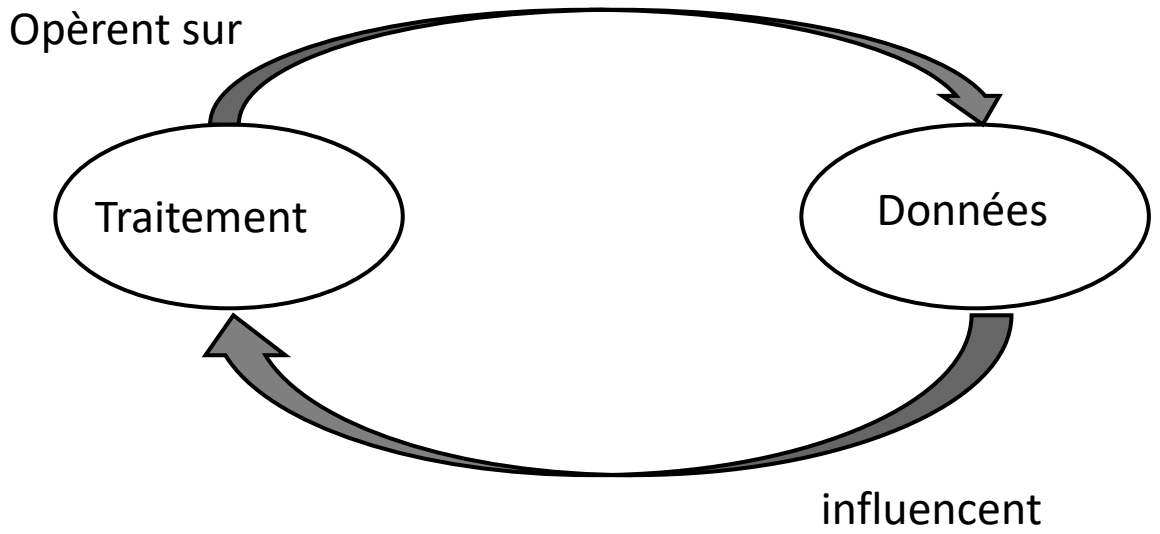
Plan

- Programmation orientée objet
- Programmation orientée objet en Java
- Classe: Objet et Encapsulation
- Héritage et Polymorphisme
- Les classes abstraites et interfaces
- Les Exceptions
- Les collections
- JDBC

La programmation procédurale vs orienté objet (1)

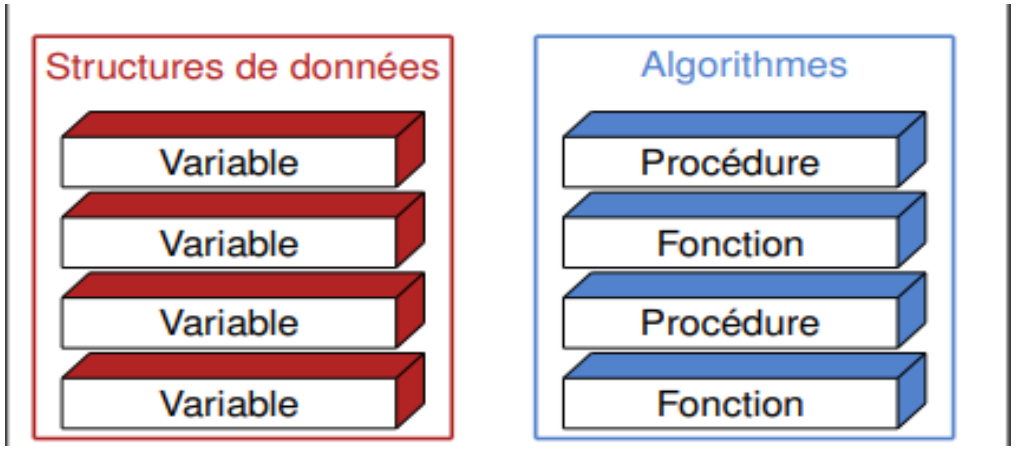
Il existe deux grands types de programmation:

- **La programmation procédurale:** consiste à diviser un programme en une série de procédures ou de fonctions.



➔ Les entités apparaissent de façon séparée.

La programmation procédurale vs orienté objet (2)



Difficulté de maintenir le code: le code est organisé en fonctions et sous-programmes qui sont appelés à partir du programme principal ➔ Cela peut rendre le code difficile à comprendre et à maintenir, car il peut être difficile de suivre la logique du programme en sautant d'une fonction à l'autre.

Difficulté à programmer en équipe : les programmes procéduraux peuvent être difficiles à comprendre et à modifier pour les autres membres de l'équipe.

La programmation procédurale vs orienté objet (3)

Complexité des programmes: les programmes procéduraux peuvent devenir très complexes et difficiles à maintenir lorsque le code devient volumineux. Les fonctions imbriquées peuvent également compliquer la compréhension du code.

Manque de modularité: la programmation procédurale n'offre pas une bonne modularité pour les programmes ➔ Ce qui rend la réutilisation de code plus difficile.

➔ Peut être difficile de réutiliser des fonctions individuelles dans d'autres programmes ou projets.

La programmation procédurale vs orienté objet (4)

- **La Programmation Orientée Objet (POO) :** est un paradigme de programmation qui vise à modéliser le monde réel en utilisant des concepts **de classes** et **d'objets.**

- **La classe:** une classe est un ensemble de code contenant des variables et des fonctions permettant de créer des objets.

  ➔Une classe peut contenir plusieurs objets.

- **Les objets:** sont des instances de classes, qui sont des modèles de données qui définissent **les propriétés** et **les comportements** des objets.

  - ✓ **Les propriétés:** sont des variables (**attributs**) qui stockent des données spécifiques à chaque objet
  - ✓ **Les comportements**: sont des fonctions (**méthodes**) qui peuvent être appelées sur les objets pour effectuer des traitements.

➔ les **attributs** et **les méthodes** d'un objets sont appelés ses **membres.**

La programmation procédurale vs orienté objet (5)

❑ Différence entre classe et Objet:

Il est important de saisir les différences entre les notions de classe et instance de la classe :

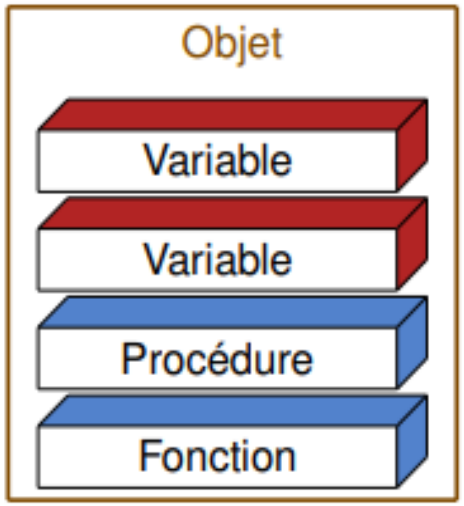
- **classe = attributs + méthodes + mécanismes d'instanciation + mécanismes de destruction**

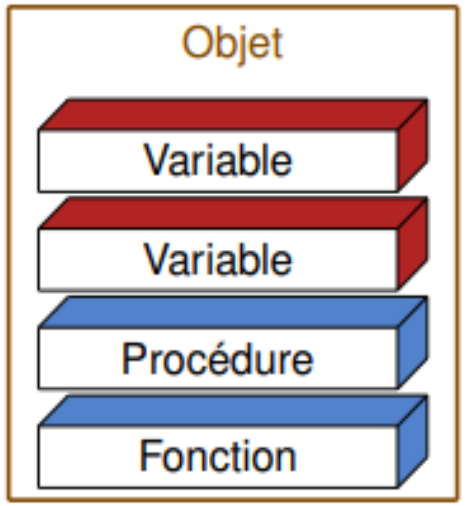
- **Instance de la classe (Objet) = valeurs des attributs + accès aux méthodes**

- L'instanciation est le mécanisme qui permet de créer des instances dont les traits sont décrits par la classe.
- La destruction est le mécanisme qui permet de détruire une instance de la classe.

➔ L'ensemble des instances d'une classe constitue l'extension de la classe .

La programmation procédurale vs orienté objet (6)





➔ Regrouper les traitements et les données dans une seule et même entités.

- La POO offre:
  - **Modularité:** C'est la capacité de diviser un système en modules distincts qui sont indépendants les uns des autres.

    ➔ ce qui facilite la compréhension du code et la maintenance du programme.

    ➔ Cela facilite le développement de grandes applications

La programmation procédurale vs orienté objet (7)

- **Réutilisabilité:** c'est la capacité de réutiliser du code existant dans d'autres parties de l'application ou dans d'autres applications.

  ➔Gagner du temps.

- **Maintenabilité:** se réfère à la facilité avec laquelle le code peut être modifié et mis à jour.

  ➔Les systèmes logiciels évoluent constamment pour s'adapter aux nouveaux besoins et aux nouvelles fonctionnalités

  ➔La POO permet une plus grande flexibilité en permettant de modifier le comportement d'un objet sans modifier le reste du système.

➔ la POO permet aux programmeurs de créer des applications plus évolutives, plus flexibles et plus faciles à maintenir sur le long terme.

La Programmation Orienté Objet (1)

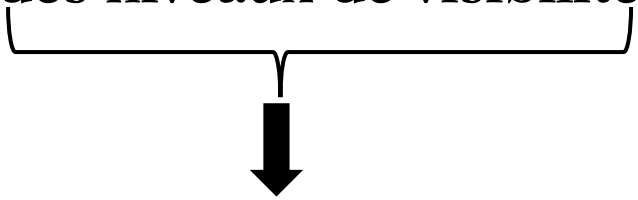
- La POO offre en réalité **quatre concepts** fondamentaux:
  - Encapsulation
  - Abstraction
  - Héritage
  - Polymorphisme

➔ Organiser des programmes complexes.

Le concept d'encapsulation (1)

❑ **L'encapsulation:**

- est un mécanisme qui consiste à regrouper dans le même objet informatique, **les données** (**les attributs**) et **les traitements** (**les méthodes/ les fonctions**).
- Permet de cacher l'information contenue dans un objet et **ne permettant l'accès qu'à des fonctionnalités spécifiques et contrôlées** ➔ ce qui contribue à **la sécurité** et à **l'intégrité** des données.
- Elle permet aussi de gérer **l'accès aux données**.
- Définir **des niveaux de visibilité** des éléments de la classe .



Ces niveaux de visibilité définissent **les droits d'accès** aux données.

Le concept d'encapsulation (2)

❑ Il existe **trois niveaux** de visibilité:

- **Public:** les membres publics peuvent être accessibles depuis n'importe où dans le code, que ce soit à l'intérieur ou à l'extérieur de la classe.
- **Protégé:** les membres protégés sont accessibles à la **classe elle-même** et à **toutes les sous-classes** qui en héritent, mais pas aux autres parties du code.
- **Privé:** les membres privés **ne sont accessibles** qu'à **la classe elle-même**.

➔ L'utilisation de ces droits d'accès permet **de contrôler l'accès** aux données de la classe, de **protéger les membres sensibles**, de **faciliter la maintenance** du code en limitant les interactions entre les différentes parties du programme, et **d'améliorer la lisibilité** et la **compréhension** du code en clarifiant les relations entre les différentes parties du programme.

Le concept d'abstraction (1)

- **L'abstraction:**
  - Est un concept qui permet de **masquer** la complexité d'un système en ne montrant que les informations essentielles à son utilisation.
  - Il vous suffit de savoir quelles méthodes de l'objet sont disponibles pour appeler et quels paramètres d'entrée sont nécessaires pour déclencher une opération spécifique. Mais vous n'avez pas besoin de comprendre comment cette méthode est implémentée et quels types d'actions elle doit effectuer pour créer le résultat attendu.

La notion d'héritage (1)

❑ **L'héritage:**

- Est une notion fondamentale en POO qui permet de **créer une nouvelle classe** à partir d'une **classe existante**.
- Permet à **une classe fille** (ou **dérivée**) de bénéficier de toutes les propriétés et méthodes de **la classe mère** (ou **base/superclasse**), tout en permettant **d'ajouter** de nouvelles propriétés et méthodes spécifiques à la classe dérivée.

➔ La classe dérivée hérite des caractéristiques de la classe de base et peut les utiliser comme si elles étaient ses propres membres.

- L'héritage permet ainsi **de réutiliser le code existant** et **de faciliter la conception** et **la maintenance de programmes** complexes en regroupant des fonctionnalités communes dans une classe de base, et en créant des classes dérivées spécialisées pour des cas d'utilisation spécifiques.

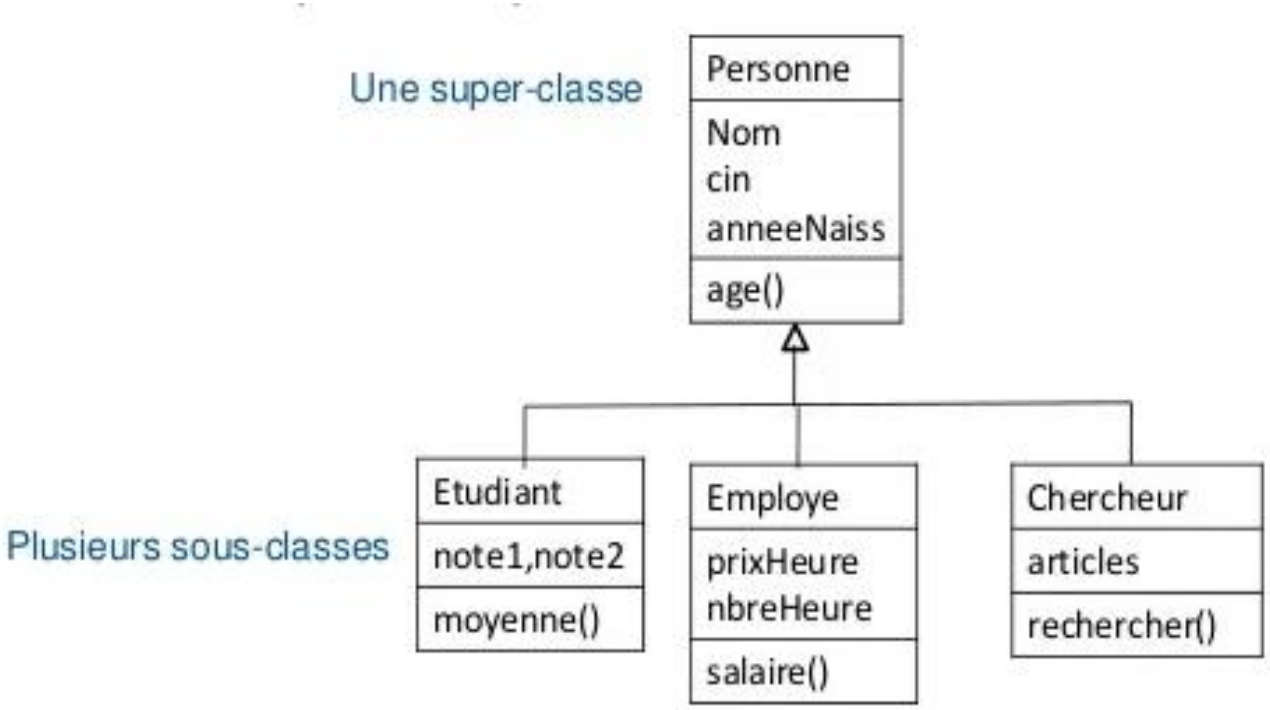
La notion d'héritage (2)

❑ **L'héritage:**

➢ Est souvent utilisé pour créer une hiérarchie de classes appelée aussi **arborescence de classes**, une relation de **parenté** existe entre les différentes classes.

➔ On peut généralement exprimer la relation qui lie une classe fille à sa mère par la phrase "**est un**" (de l'anglais "**is a**").

❑ **Exemple:**



La notion d'héritage (3)

❑ **Remarque:**

Certains langages orientés objet permettent de faire **de l'héritage multiple**

- Ce qui signifie qu'ils offrent la possibilité de faire hériter **une classe de plusieurs superclasses**.
- Cette technique permet de regrouper au sein d'une seule et même classe les attributs et méthodes de plusieurs classes.

➔ C++ permet l'héritage multiple mais Java ne le permet pas.

Le polymorphisme (1)

❑ Le polymorphisme:

- Le nom de **polymorphisme** signifie **qui peut prendre plusieurs formes**. Cette caractéristique est un des concepts essentiels de la POO.

- Il y a **deux types** de polymorphisme en POO :
  - **La surdéfinition ou surcharge** (en anglais overloading)
  - **La redéfinition** (en anglais overriding)

❑ La surdéfinition ou surcharge:

- Permet à une méthode d'avoir des comportements différents en fonction des paramètres qu'elle reçoit. ➔ consiste à définir plusieurs fonctions **de même nom** mais possédant **des paramètres différents** (**en nombre** et/ou **en type**).
- Le choix de **la bonne méthode** à adopter se fait **automatiquement** par le **compilateur** en fonction du type de donnée passée en paramètre.

Le polymorphisme (2)

❑ Exemple:

- La méthode **int addition(int, int)** qui retourne la somme de deux entiers.
- La méthode **float addition(float, float)** qui retourne la somme de deux flottants.
- La méthode **char addition(char, char)** qui retourne la somme de deux caractères.

❑ La redéfinition:

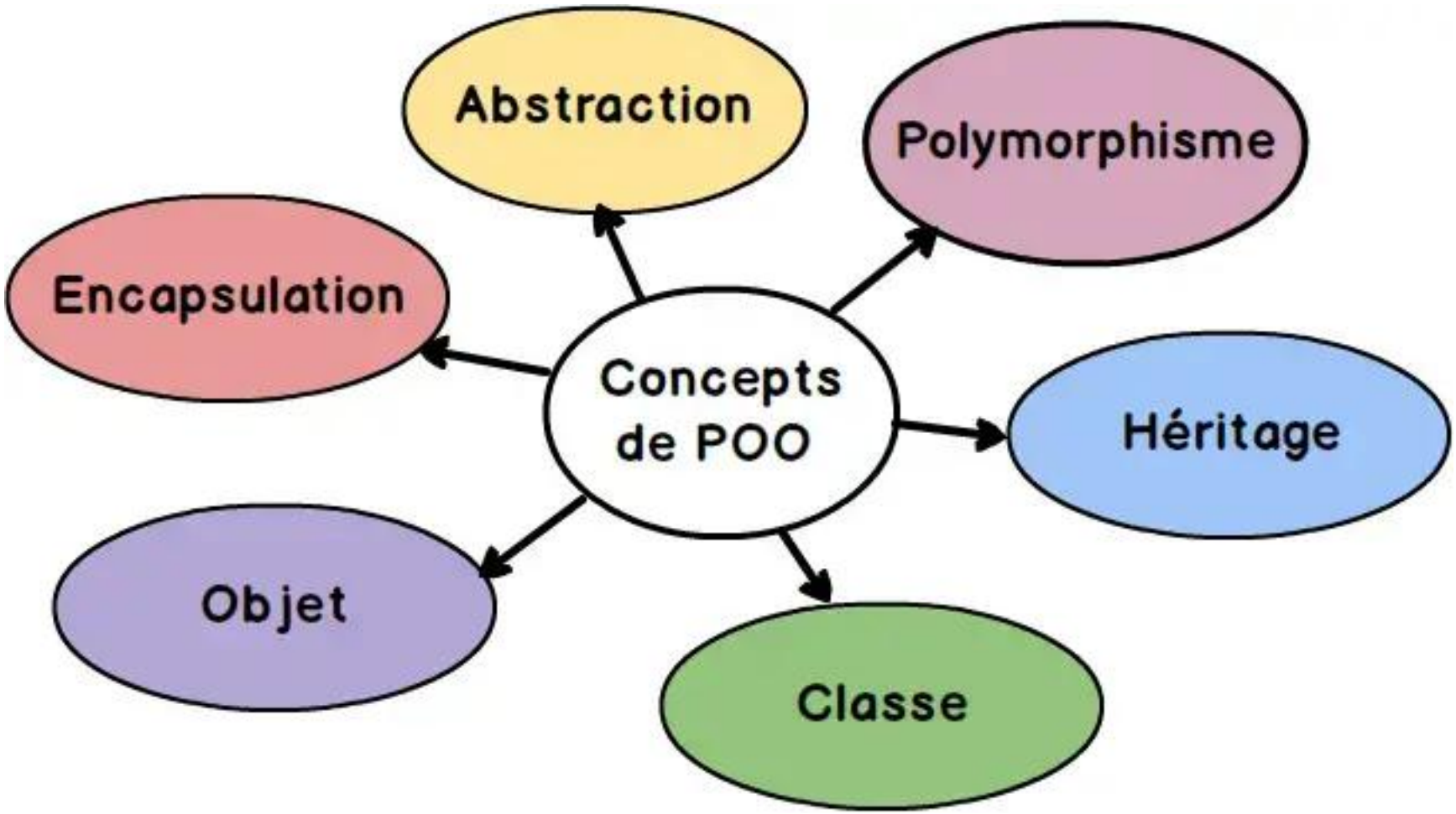
- Consiste à implémenter **de nouveau**, dans une classe fille, **la même méthode** qui a été définie dans sa classe mère.
- La nouvelle méthode doit **avoir exactement le même nom** et **la même signature** (paramètres) que celle définie dans la classe mère.

Le polymorphisme (3)

❑ **Exemple:**

- Une classe **Animal** a une méthode **manger()** et que plusieurs classes comme **Chien**, **Chat**, **Oiseau** héritent de la classe Animal
- Alors chaque sous-classe pourra réimplémenter la méthode **manger()** selon ses besoins spécifiques.
- Lors de l'exécution, en appelant **la méthode manger()** sur un objet **de type Animal**, la méthode adéquate sera choisie en fonction de son type réel (Chien, Chat, Oiseau, etc.).

Résumé


Abstraction
Polymorphisme
Encapsulation
Concepts de POO
Héritage
Objet
Classe